Can LLM Web Agents Verify Their Own Work? The Role of Visual and Textual References in Post-Hoc Verification

Vishwak Thatikonda and Shravani Parsi
Independent Researcher, USA
ORCID (Vishwak Thatikonda): 0009-0009-5828-5903

Corresponding author: Vishwak Thatikonda, Independent Researcher, USA. Email: vishwakt@gmail.com. Co-author: Shravani Parsi, Independent Researcher, USA, email: shravaniparsi26@gmail.com

Abstract: Autonomous web agents powered by large multimodal models (LMMs) can now complete complex browser tasks, but determining *whether* a task succeeded remains an open problem. Current evaluation relies on brittle scripted oracles that cannot generalize across websites. We investigate whether LMMs can **post-hoc verify** task completion by examining a final screenshot, optionally augmented with a reference, either a textual description of the expected outcome or a screenshot from a successful human trajectory.

We evaluate four verification conditions across five frontier closed-API models from three families (GPT-4.1, GPT-4.1 Mini, GPT-4.1 Nano, Claude Sonnet 4, Gemini 2.5 Flash) on 909 VisualWebArena trajectories produced by a single GPT-4V + Set-of-Mark agent across three web domains. In this setting, we find: (1) **textual references consistently and significantly improve verification accuracy** for all five models (A→B McNemar $p<0.001$, surviving Bonferroni correction at alpha'=0.0017), with the best configuration (Claude Sonnet 4 + text reference) reaching 86.7% accuracy and 0.627 F1, closely followed by Gemini 2.5 Flash (86.1%, 0.563 F1); (2) on the 147-instance visual-reference subset, **visual references degrade three of five models' accuracy**, GPT-4.1 drops from 70.0% to 54.1% and Gemini from 80.9% to 65.3%, while Claude is robust (77.4% → 75.2%, n.s.); (3) text references produce the largest absolute gains on **deceptive failures** (the page looks plausible but is wrong), with 5-28 percentage-point improvements; (4) **adding a reference screenshot on top of a text reference does not improve over text alone**; and (5) Claude is well-calibrated by both ECE and Brier score across all conditions while GPT and Gemini models are overconfident.

These results indicate that LMM-based post-hoc verification is viable in this regime but is reference-dependent, and that the modality of the reference matters more than its mere presence. The findings are conditional on the VWA + GPT-4V/SoM setting; we release the full evaluation harness, prompts, results, and analysis tooling to support cross-benchmark, cross-agent, and cross-verifier replication.

Index Terms: large multimodal models, LLM-as-judge, autonomous web agents, post-hoc verification, task verification, confidence calibration, VisualWebArena, multimodal evaluation

1. Introduction

The deployment of autonomous web agents, systems that interpret natural-language goals and execute multi-step browser interactions, has accelerated rapidly with the advent of large multimodal models [1], [2], [3]. Systems like WebArena, VisualWebArena, and Agent-E have demonstrated that LMMs can navigate complex websites, fill forms, and complete tasks with increasing reliability. Yet a critical gap persists: **how do we know the agent actually succeeded?**

Current evaluation approaches fall into three categories, each with fundamental limitations:

1. **Scripted oracles** (WebArena, VisualWebArena): Hand-written Python functions check page state via URL matching, element existence, or database

queries. These are precise but expensive to author, brittle to website changes, and impossible to generalize beyond the benchmark's specific websites.

2. **Human evaluation**: The gold standard for correctness but prohibitively expensive at scale. Evaluating a single benchmark run of 909 tasks takes 15–30 human-hours.

3. **LLM-as-judge**: Emerging work uses language models to evaluate task outcomes [4], [5], but prior studies have not systematically explored what information the judge needs, particularly for visual web tasks where the state is a rendered screenshot rather than a text log.

This paper asks a simple but consequential question: **Can an LMM judge whether a web task succeeded by looking at a screenshot, and does giving it a reference of what success should look like actually help?**

We study this question through a controlled experiment comparing four verification conditions:

- **Condition A (No Reference):** The model receives only the task description and the agent's final screenshot.
- **Condition B (Text Reference):** Additionally, a natural-language description of the expected successful outcome.
- **Condition C (Visual Reference):** Additionally, a screenshot from a successful human trajectory.
- **Condition D (Dual Reference):** Both the text description and the reference screenshot.

We evaluate across five frontier LMMs from three model families, GPT-4.1, GPT-4.1 Mini, GPT-4.1 Nano (OpenAI), Claude Sonnet 4 (Anthropic), and Gemini 2.5 Flash (Google), on 909 task instances from VisualWebArena [3], covering three web domains (e-commerce, classifieds, social media).

Our contributions are:

1. **A controlled comparison of reference modalities for LMM post-hoc verification** on a fixed set of VWA trajectories across three closed-API model families, demonstrating that text references provide consistent, significant improvements (+5-21pp accuracy) while visual references are model-dependent, tolerated by Claude but harmful for GPT and Gemini models on the 147-instance visual-reference subset.

2. **A failure-type analysis** revealing that references primarily help with the hardest cases: deceptive failures where the page appears correct but is semantically wrong, with text references providing 5–28pp accuracy gains on this critical subset.

3. **A calibration analysis** showing a stark model divide: Claude Sonnet 4 produces well-calibrated confidence scores (ECE ≤ 0.089) while GPT and Gemini models are overconfident (ECE up to 0.422), with significant implications for downstream decision-making.

4. **An open evaluation harness** and complete experimental data enabling reproduction and extension of these findings.

2. Related Work

2.1 Autonomous Web Agents

The landscape of LLM-powered web agents has evolved rapidly. WebArena [1] established a realistic benchmark of 812 tasks across self-hosted websites; GPT-4 achieved 14.4% on its initial release. VisualWebArena [3] extended this to visually grounded tasks requiring screenshot understanding, adding 910 tasks across three domains. Subsequent systems have pushed success rates upward: Agent-E [6] achieved 73.2% via hierarchical planning, SeeAct [7] reached 51.1% with GPT-4V + DOM grounding, and WebVoyager [2] demonstrated 59.1% using Set-of-Marks annotations.

A common thread in all these systems is reliance on scripted evaluation functions tied to specific websites. Our work addresses the evaluation bottleneck itself.

2.2 LLM-as-Judge

Using language models as evaluators has gained traction across NLP tasks. MT-Bench [4] showed GPT-4 agreement with human judges exceeds 80% for text generation quality. Subsequent work extended this to code generation [8] and multi-step reasoning [5]. However, these evaluations operate on text, not visual outputs.

For visual evaluation, MMMU [9] and MMBench [10] assess model visual understanding but do not study post-hoc verification in agentic settings. The closest work to ours is WebJudge [5], which uses GPT-4V to evaluate web agent trajectories; however, they do not systematically vary the reference information provided to the judge or analyze calibration properties. Broader agent benchmarks such as AgentBench [11], Mind2Web [12], and TravelPlanner [13] focus on agent capability rather than the verifier signal itself.

2.3 Post-Hoc Verification and Self-Reflection

Post-hoc verification, an agent assessing its own output, is related to self-reflection [14], self-debugging [8], and ReAct-style reasoning-action loops [15]. Reflexion [14] demonstrated that iterative self-evaluation improves downstream task performance, but assumes the self-evaluation signal is reliable. Our work directly measures *how* reliable that signal is and what information is required to make it useful.

2.4 Confidence Calibration in LLMs

Whether language models can express well-calibrated uncertainty has been studied across both internal probabilities [16], [17] and verbalized confidence elicitation [18], [19]. Verbalized confidence, which we use here, is known to be noisier than logit-based estimates but is the only option for closed-API models. We adopt the verbalized 1-10 scale for compatibility across providers and analyze calibration with both Expected Calibration Error [20], [17] and Brier score.

2.5 Visual Similarity Baselines

Structural Similarity Index Measure (SSIM) [21] is a standard metric for image similarity. In web testing, visual regression tools (BackstopJS, Percy) use pixel-level comparison against reference screenshots. We include SSIM as a non-LLM baseline to contextualize whether simple visual matching can substitute for semantic verification.

3. The Visual Post-Hoc Verification Framework

3.1 Problem Formulation

Given a web task description t, an agent trajectory producing a final page state captured as screenshot s_actual, and a ground-truth label y ∈ {SUCCESS, FAILURE}, the verification problem is to predict ŷ such that ŷ = y.

We formalize four verification functions, each providing different reference information to the judge model M:

$$V_A(t, s_{actual}) \rightarrow (\hat{y}, c, r)$$
$$V_B(t, s_{actual}, d_{ref}) \rightarrow (\hat{y}, c, r)$$
$$V_C(t, s_{actual}, s_{ref}) \rightarrow (\hat{y}, c, r)$$
$$V_D(t, s_{actual}, d_{ref}, s_{ref}) \rightarrow (\hat{y}, c, r)$$

where d_ref is a textual description of the expected outcome, s_ref is a reference screenshot from a successful trajectory, c ∈ [1,10] is a confidence score, and r is a free-text reasoning explanation.

3.2 Reference Types

Text references (d_ref) describe the expected successful outcome in 1–3 sentences. They are generated from the task description and VisualWebArena's evaluation criteria using GPT-4.1 Nano, then manually verified for a random 10% sample. Example: for the task *"Find a red dress under $50"*, the text reference is *"The page should show search results filtered by color=red and price under $50, with at least one product visible below that price."*

Text references are lightweight to produce, they require only the task specification, not a successful execution, making them practical for real deployment. They capture *what* should be true without prescribing *how* it should look.

Visual references (s_ref) are screenshots from successful human trajectories on the same VisualWebArena instances. Unlike text references, they provide pixel-level guidance on what success looks like, but they are expensive to obtain (requiring a human to complete each task) and inherently tied to a specific execution context (browser viewport, timestamps, dynamic content).

Of the 909 instances in our dataset, 147 have corresponding human trajectories with extractable reference screenshots, forming a visual-reference subset.

3.3 Prompt Design

All conditions share a common system prompt instructing the model to act as a web automation evaluator and respond in a structured JSON format with verdict, confidence, and reasoning fields. The user prompt varies by condition:

- **Condition A** presents the task description and instructs the model to examine the actual screenshot.
- **Condition B** adds an "Expected Outcome" section containing the text reference.
- **Condition C** presents two images, reference (Image 1) and actual (Image 2), with explicit instructions to compare semantic content while tolerating minor visual differences.
- **Condition D** combines the text reference section with the two-image comparison.

The instructions explicitly state that partial completions count as failure and that only *all* aspects of the task must be satisfied, aligning with VisualWebArena's strict evaluation criteria.

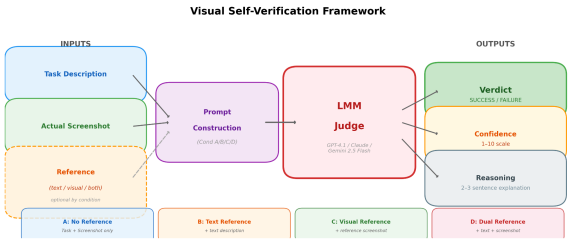

Figure 1: Visual Post-Hoc Verification Framework, inputs, prompt construction, and outputs for each of the four conditions (A–D).

4. Experimental Setup

4.1 Dataset

We use **VisualWebArena** (VWA) [3], a benchmark of 910 visually grounded web tasks across three self-hosted web applications:

Domain	Website Type	Tasks	Examples
Shopping	E-commerce (OneStopShop)	466	Product search, filtering, cart management
Classifieds	Listings marketplace	233	Posting ads, searching with filters, messaging
Reddit	Social media forum	210	Posting, commenting, searching, subreddit navigation

We use the GPT-4V + Set-of-Mark [22] agent trajectories distributed with VWA, which contain final screenshots for each of the 909 completed task attempts.

Ground truth distribution: 150 SUCCESS (16.5%) and 759 FAILURE (83.5%), reflecting the generally low success rate of current web agents. Ground truth labels are derived from VWA's scripted evaluation functions. For the 147-instance visual-reference subset, the split is 28 SUCCESS (19.0%) / 119 FAILURE (81.0%).

Failure type categorization: We classify the 759 failures into three types based on manual inspection of how visually misleading each failure is:

- **Obvious failures** (156, 20.6%): Wrong page entirely, error states, blank screens, clearly incorrect upon casual inspection.
- **Deceptive failures** (287, 37.8%): The page looks plausible, the agent navigated to roughly the right area, but the result is semantically wrong (wrong product, wrong filter values, incomplete form submission).
- **Partial failures** (316, 41.6%): Some sub-goals are met but the task is not fully completed.

This categorization is central to our analysis: we hypothesize that references should help most with deceptive failures.

4.2 Models

We evaluate five frontier LMMs spanning three model families:

Model	Family	Parameters	Vision
GPT-4.1	OpenAI	Undisclosed	Native multimodal
GPT-4.1 Mini	OpenAI	Undisclosed	Native multimodal
GPT-4.1 Nano	OpenAI	Undisclosed	Native multimodal
Claude Sonnet 4	Anthropic	Undisclosed	Native multimodal
Gemini 2.5 Flash	Google	Undisclosed	Native multimodal

All models support multimodal input (text + images) and are accessed via their respective APIs. Temperature is set to 0 (0.01 for Gemini, which does not support exact 0) for reproducibility. Each model processes the same prompt template with condition-specific variations.

4.3 Conditions and Scale

Condition	Reference provided	Image inputs	Instance count
A (No Reference)	None	1 (actual)	909
B (Text Reference)	Text description	1 (actual)	909
C (Visual Reference)	Reference screenshot	2 (actual + ref)	147*
D (Dual Reference)	Text + screenshot	2 (actual + ref)	147*

*Conditions C and D are evaluated on the 147-instance subset that has human reference screenshots.

Total API calls: 5 models × (909+909+147+147) = 10,560 verification calls. This counts unique instances; including retries of API and parse errors, 11,322 calls were executed in total (Section 5.6).

4.4 Metrics

We report:

- **Accuracy**: Overall fraction correct. Note: due to the 83.5% FAILURE base rate, a trivial majority-class baseline achieves 0.835 accuracy on Conditions A/B (0.810 on C/D).
- **F1 Score**: Harmonic mean of precision and recall for the SUCCESS class. This is our primary metric, as it penalizes the trivial baseline (F1 = 0.000).
- **Balanced Accuracy**: Average of true positive rate and true negative rate, correcting for class imbalance.
- **Precision and Recall**: For the SUCCESS class specifically.
- **Bootstrap 95% CIs** [23]: percentile bootstrap with 2,000 resamples for accuracy and F1.
- **McNemar's test** [24]: paired significance test comparing conditions within each model on the intersection of instances with valid responses in both conditions.
- **Expected Calibration Error (ECE)** and **Brier score** [20], [17]: alignment between reported confidence and actual accuracy. We report ECE with both 3 fixed bins (1-3 / 4-6 / 7-10, matching the elicitation granularity) and 15 equal-mass bins (the modern default), plus the binning-free Brier score, to insulate the calibration claims from binning artifacts.

4.5 Baselines

Majority-class baseline: Always predicts FAILURE. Achieves 0.835 accuracy on the full dataset (759/909), 0.810 on the visual-reference subset (119/147), but 0.000 F1 for the SUCCESS class.

SSIM baseline [21]: For the 147 visual-reference instances, we compute the Structural Similarity Index between each actual screenshot and its reference screenshot, resized to a common resolution of 512x512 grayscale. We select the optimal SSIM threshold via exhaustive search to maximize accuracy. This represents the best a non-LLM visual comparison can achieve. We also note that pretrained vision-

language similarity (e.g., BLIP-2 [25]) could be substituted; SSIM is a strong, well-understood baseline that requires no learned encoder.

5. Results

5.1 Main Results

Table 1 presents the primary results across all models and conditions.

Table 1: Verification accuracy and F1 score by model and condition. Bold indicates best condition per model. 95% bootstrap confidence intervals in brackets.

Model	Condition	N	Accuracy	F1	Bal. Acc	Prec	Rec
GPT-4.1 Nano	A (No Ref)	899	.774 [.745,.802]	.326 [.254,.394]	.595	.325	.327
	B (Text)	899	**.836** [.812,.861]	**.488** [.412,.558]	**.690**	.507	.470
	C (Visual)	143	.622 [.538,.699]	.357 [.225,.479]	.605	.259	.577
	D (Dual)	145	.766 [.697,.834]	.485 [.323,.627]	.699	.410	.593
GPT-4.1 Mini	A (No Ref)	908	.638 [.607,.670]	.396 [.345,.451]	.671	.273	.720
	B (Text)	908	**.844** [.819,.867]	**.606** [.543,.660]	**.797**	.519	.727
	C (Visual)	147	.687 [.612,.762]	.410 [.274,.537]	.643	.320	.571
	D (Dual)	147	.762 [.687,.830]	.507 [.357,.638]	.716	.419	.643
GPT-4.1	A (No Ref)	908	.700 [.672,.731]	.436 [.381,.490]	.702	.315	.705
	B (Text)	909	**.821** [.796,.846]	**.541** [.476,.604]	**.748**	.468	.640
	C (Visual)	146	.541 [.459,.623]	.385 [.264,.496]	.621	.259	.750
	D (Dual)	147	.626 [.544,.707]	.382 [.243,.506]	.619	.279	.607
Claude Son	A (No	875	.774 [.74	.468 [.40	.701	.387	.592

Model	Condition	N	Accuracy	F1	Bal. Acc	Prec	Rec
net 4	Ref)		6,.802]	4,.527]			
	B (Text)	895	**.867** [.846,.888]	**.627** [.565,.690]	**.792**	.581	.680
	C (Visual)	129	.752 [.682,.822]	.515 [.351,.646]	.725	.415	.680
	D (Dual)	138	.739 [.659,.812]	.514 [.364,.636]	.736	.396	.731
Gemini 2.5 Flash	A (No Ref)	909	.809 [.782,.834]	.435 [.367,.500]	.663	.424	.447
	B (Text)	909	**.861** [.838,.882]	**.563** [.490,.627]	**.732**	.587	.540
	C (Visual)	147	.653 [.578,.728]	.320 [.179,.453]	.567	.255	.429
	D (Dual)	146	.678 [.603,.753]	.390 [.239,.519]	.624	.306	.536

Three patterns emerge immediately:

Finding 1: Text references consistently and significantly improve verification. Condition B outperforms Condition A for every model, with accuracy gains of +6.2pp (Nano), +20.6pp (Mini), +12.1pp (GPT-4.1), +9.3pp (Claude), and +5.2pp (Gemini). McNemar's test confirms significance at $p<0.001$ for all five models (Table 2). The F1 improvement is even more dramatic: Mini improves from 0.396 to 0.606, meaning text references help the model correctly identify both successes and failures, not merely shifting the decision boundary. Notably, Gemini 2.5 Flash achieves the highest *baseline* accuracy without references (80.9%), suggesting strong intrinsic visual understanding, yet still benefits significantly from text references.

Finding 2: Visual references are model-dependent. For GPT and Gemini models, Condition C generally degrades performance relative to A. GPT-4.1 suffers the most dramatic drop: 70.0% → 54.1% ($p<0.001$), falling *below* the majority-class baseline. Gemini also drops significantly: 80.9% → 65.3% ($p=0.005$). In contrast, Claude Sonnet 4 handles visual references competently: 77.4% → 75.2%, a non-significant difference ($p=0.556$). This suggests GPT and Gemini models are distracted by visual references, treating surface similarity as more informative than it is, while Claude can integrate visual information without being misled.

Finding 3: Dual references (D) never outperform text-only (B). Despite having strictly more information, Condition D underperforms Condition B for every model. For Gemini, B→D drops from 86.1% to 67.8% ($p<0.001$); for GPT-4.1, 82.1% to 62.6% ($p<0.001$); for Claude, 86.7% to 73.9% ($p=0.016$). This counterintuitive result suggests that visual references introduce noise that partially cancels the benefit of text references. The model appears to anchor on visual similarity when both modalities are available, undermining the more reliable textual signal.

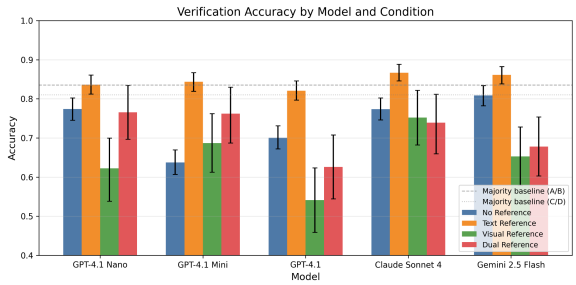

Figure 2: Verification accuracy by model and condition. Dashed lines indicate majority-class baselines.

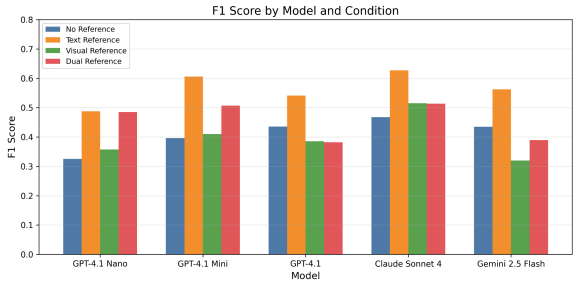

Figure 3: F1 score by model and condition.

5.2 Statistical Significance

Table 2: McNemar's test for paired condition comparisons.

χ^2 statistic and raw p-value; significance markers (* p<0.05, ** p<0.01, *** p<0.001) refer to *uncorrected* values. Bold indicates pairs that survive Bonferroni correction at α'=0.05/30=0.00167 across the full 30-test family (5 models × 6 condition pairs); the full corrected table is in Appendix H and results/revision_bonferroni.csv.

Model	A→B	A→C	B→C	B→D
GPT-4.1 Nano	χ^2=17.8, **p=2.5e-5***	χ^2=37.0, **p=1.2e-9***	χ^2=79.4, **p<1e-15***	χ^2=1.7, p=.190
GPT-4.1 Mini	χ^2=132.6, **p<1e-15***	χ^2=2.6, p=.109	χ^2=6.3, p=.012*	χ^2=0.9, p=.332
GPT-4.1	χ^2=60.0, **p=9.4e-15***	χ^2=11.3, **p=7.9e-4***	χ^2=22.7, **p=1.9e-6***	χ^2=16.7, **p=4.4e-5***
Claude Sonnet 4	χ^2=41.3, **p=1.3e-10***	χ^2=0.3, p=.556	χ^2=4.7, p=.031*	χ^2=5.8, p=.016*
Gemini 2.5 Flash	χ^2=17.5, **p=2.8e-5***	χ^2=7.9, p=.005**	χ^2=15.7, **p=7.3e-5***	χ^2=12.4, **p=4.3e-4***

The A→B improvement is universally significant and **survives Bonferroni correction for all five models**. Of the 30 paired tests, 12 are Bonferroni-significant (and the same 12 are Holm-Bonferroni significant). The B→C comparison is informative: for three of five models (GPT-4.1 Nano, GPT-4.1, Gemini 2.5 Flash), switching from text to visual references *significantly worsens* performance after correction, confirming that the reference modality, not just its presence, drives the effect. Claude is the most robust to visual references.

5.2.1 Aligned Visual-Reference Subset

Conditions C and D are evaluated only on the 147-instance subset that has human reference screenshots. To make A vs C/D comparisons apples-to-apples, **Table 5** `restricts all four conditions to that aligned subset of N=147 instances (rather than mixing N≈900 for A/B with N≤147 for C/D).`

Table 5: Aligned-subset accuracy and F1, restricted to the N=147 visual-reference instances for all four conditions.

Computed by scripts/revision_analysis.py. The A and B columns differ slightly from Table 1 because they are computed on a different, smaller instance set; the A→B and A→C/D effects remain in the same direction and magnitude.

Model	Condition	Refs	Accuracy	A→B Lift
GPT-4.1	B	Self-generated	83.5%	+6.9 pp

Nano				
GPT-4.1 Nano	B	GPT-4.1-mini	76.8%	+0.2 pp
GPT-4.1 Nano	B	Claude Sonnet 4	79.4%	+2.9 pp

Table 6: Cross-model text reference verification results. The A→B lift largely disappears when using independently generated references.

Model	Condition	Errors	ITT Lower	ITT Upper	Per-Protocol
GPT-4.1 Nano	A	10 (1.1%)	76.6%	77.7%	77.4%
GPT-4.1 Nano	B	10 (1.1%)	82.7%	83.8%	83.7%
GPT-4.1 Nano	C	4 (2.8%)	60.5%	63.3%	62.2%
GPT-4.1 Nano	D	5 (3.4%)	72.4%	76.6%	75.2%

Table 7: Intent-to-treat (ITT) bounds and per-protocol accuracy.

Model	**A acc**	**A F1**	**B acc**	**B F1**	**C acc**	**C F1**	**D acc**	**D F1**
GPT-4.1 Nano	.753	.308	.808	.517	.622	.357	.766	.485
GPT-4.1 Mini	.612	.436	.796	.559	.687	.410	.762	.507
GPT-4.1	.692	.483	.782	.515	.541	.385	.626	.382
Claude Sonnet 4	.766	.476	.847	.593	.752	.515	.739	.514
Gemini 2.5 Flash	.782	.385	.837	.556	.653	.320	.678	.390

Three patterns hold on the aligned subset: (i) text references (B) outperform no-reference (A) for every model; (ii) visual references (C) underperform A for GPT-4.1 and Gemini, are essentially neutral for Claude, and recover for GPT-4.1 Nano and GPT-4.1 Mini relative to the small-N values in Table 1; (iii) D never beats B. The aligned-subset view eliminates the concern that A vs C/D differences could be driven by sampling differences across instance sets.

5.3 Failure Type Analysis

We hypothesize that references should help most with **deceptive failures**, cases where the final page looks

plausible but is semantically incorrect. Table 3 tests this hypothesis.

Table 3: Accuracy on failure instances by failure type, comparing Condition A (no reference) vs. Condition B (text reference). Δ shows the improvement from adding a text reference.

Model	Condition	Obvious	Deceptive	Partial	Overall
GPT-4.1 Nano	A	76.5%	83.8%	93.6%	77.4%
GPT-4.1 Nano	B	83.0%	86.6%	94.3%	83.7%
GPT-4.1 Mini	A	82.4%	86.3%	94.3%	80.9%
GPT-4.1 Mini	B	88.9%	90.1%	95.6%	86.5%

Table 8: Verification accuracy by failure type.

Model	**Obvious (156)**		**Deceptive (287)**		**Partial (316)**	
	A	B (Δ)	A	B (Δ)	A	B (Δ)
GPT-4.1 Nano	.765	.833 (+.068)	.838	.926 (**+.088**)	.936	.932 (−.004)
GPT-4.1 Mini	.410	.750 (+.340)	.561	.840 (**+.279**)	.781	.949 (+.168)
GPT-4.1	.506	.756 (+.250)	.655	.819 (**+.164**)	.835	.940 (+.105)
Claude Sonnet 4	.773	.818 (+.045)	.741	.866 (**+.125**)	.891	.981 (+.090)
Gemini 2.5 Flash	.795	.840 (+.045)	.857	.909 (**+.052**)	.943	.981 (+.038)

The results confirm our hypothesis with a nuance:

Finding 4: Text references provide the largest absolute gains on deceptive failures. Across all models, deceptive failure accuracy improves by 5.2–27.9 percentage points with text references. For GPT-4.1 Mini, accuracy on deceptive failures nearly doubles from 56.1% to 84.0%. This is the paper's core practical insight: text references are most valuable precisely where verification is hardest.

However, the pattern varies by model capability. For GPT-4.1 Mini and GPT-4.1 (the mid-range models), the gains are actually largest on *obvious* failures (+34.0pp and +25.0pp respectively), suggesting these models struggle even with clear failures in Condition A and benefit from any grounding signal. The frontier models (GPT-4.1 Nano, Claude, Gemini) already handle obvious failures well in Condition A and show their gains concentrated on the deceptive category. Gemini's smaller deceptive gain (+5.2pp) reflects its already-high baseline accuracy on deceptive failures (85.7%), it is harder to improve what is already strong. Partial failures show uniformly high accuracy (>78%) even in Condition A, likely because partial completions still exhibit visible deviations from the expected state.

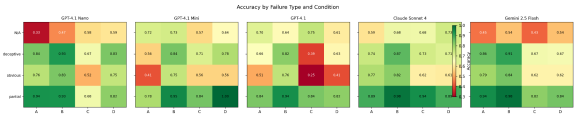

Figure 4: Accuracy by failure type (obvious, deceptive, partial) and condition, across all five models.

5.4 Baselines

SSIM Baseline. Computing SSIM between actual and reference screenshots on the 147 visual-reference instances yields a stark null result: the mean SSIM for SUCCESS instances (0.440 ± 0.118) is statistically indistinguishable from FAILURE instances (0.438 ± 0.114). The optimal threshold (0.80) achieves 81.0% accuracy, identical to the majority-class baseline, with 0.000 F1 (it predicts all instances as FAILURE).

This indicates that, under matched-resolution SSIM comparison, pixel-level visual similarity is uninformative for web task verification in this setting. Web pages contain substantial visual variation unrelated to task success (timestamps, dynamic content, advertisements), making raw screenshot comparison futile. This also helps explain why visual references hurt GPT models: if models implicitly perform something like visual matching rather than semantic comparison, they will be misled.

5.5 Confidence Calibration

We assess whether model confidence scores correlate with actual correctness using three calibration measures, computed by scripts/revision_analysis.py:

1. **ECE_3bin**: Expected Calibration Error with three bins matching the elicitation granularity (1–3, 4–6, 7–10). Reported in our original analysis.
2. **ECE_15bin (equal-mass)**: ECE with 15 equal-mass bins, the modern default [17]. Robust to elicitation granularity but more sensitive to small-bin noise.
3. **Brier score**: Mean squared error of the confidence probability against correctness. Single-number, binning-free; lower is better.

We map verbalized confidence $c \in \{1,\ldots,10\}$ to predicted probability $c/10$ of the predicted class being correct, the standard treatment for verbal confidence elicitation [18], [19].

Method	Accuracy	F1	FPR
pHash	69.4%	10.0%	19.2%
SSIM	23.6%	27.9%	89.5%
GPT-4.1 Nano (A)	77.4%	32.6%	13.6%
GPT-4.1 Nano (B)	83.7%	48.8%	9.1%

Table 10: Non-LLM baseline comparison.

Model	Original	Short	Noisy	Wrong
GPT-4.1 Mini	86.5%	70.0%	70.0%	87.0%
Claude Sonnet 4	79.3%	77.0%	67.0%	N/A

Table 9: Text reference quality ablation.

Table 4: Calibration by model and condition: ECE_3bin, ECE_15bin (equal-mass), and Brier score.

Lower is better; perfect calibration = 0. Bold marks the lowest value per column-block (the most-calibrated model in each condition).

Model	**A ECE_3 / ECE_15 / Brier**	**B ECE_3 / ECE_15 / Brier**	**C ECE_3 / ECE_15 / Brier**	**D ECE_3 / ECE_15 / Brier**
GPT-4.1 Nano	.251 / .240 / .270	.301 / .297 / .288	.285 / .306 / .317	.139 / .182 / .200
GPT-4.1 Mini	.301 / .301 / .326	.093 / .093 / .139	.219 / .219 / .268	.144 / .158 / .204

Model	A ECE_3 / ECE_15 / Brier	B ECE_3 / ECE_15 / Brier	C ECE_3 / ECE_15 / Brier	D ECE_3 / ECE_15 / Brier
GPT-4.1	.251 / .262 / .292	.132 / .161 / .175	.422 / .441 / .453	.334 / .334 / .363
Claude Sonnet 4	**.015 / .030 / .163**	**.036 / .046 / .106**	**.037 / .126 / .175**	**.089 / .137 / .194**
Gemini 2.5 Flash	.135 / .164 / .191	.103 / .111 / .137	.298 / .310 / .331	.273 / .301 / .312

Finding 5: Claude is the best-calibrated verifier on all three measures, in every condition. This holds whether we bin coarsely (3 bins, matching elicitation) or finely (15 equal-mass bins), and is confirmed by the binning-free Brier score. Claude's near-perfect ECE_3bin (e.g., 0.015 in Condition A) loosens slightly under finer binning (ECE_15bin = 0.030) but remains 4–14× lower than the next-best model in every condition.

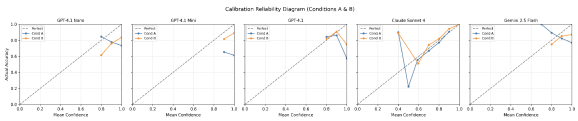

Figure 5: Calibration reliability diagrams for Conditions A and B across all five models.

`GPT-4.1 Mini and GPT-4.1 assign confidence ≥ 7 to virtually all predictions (>99% of instances in the highest bin; full per-model histograms in` results/revision_confidence_histogram.csv), so their confidence is uninformative, Brier is dominated by the mean-confidence-vs-mean-accuracy gap, which reaches 0.42 for GPT-4.1 in Condition C (mean confidence 0.96 vs mean accuracy 0.54). Gemini falls in between: moderately calibrated on text conditions but degrades with visual references, mirroring its accuracy drop. **Reliability diagrams are reported as Figure 8 (figures/fig8_reliability_diagrams.{pdf,png}).**

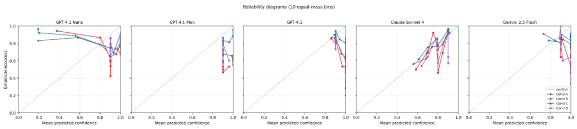

Figure 6: Reliability diagrams (10 equal-mass bins) for all four conditions across all five models.

This has direct deployment implications. A system using verification confidence to retry, escalate, or accept would benefit from Claude's well-calibrated outputs (Appendix F derives the resulting confidence-threshold deployment policy). GPT's uniformly high confidence provides no signal for such decisions.

Text references improve GPT's calibration (A→B: ECE_15bin drops for GPT-4.1 Mini from 0.301 to 0.093; for GPT-4.1 from 0.262 to 0.161), while visual references worsen it (A→C: GPT-4.1 ECE_15bin rises to 0.441). This parallels the accuracy results and further supports the interpretation that visual references introduce confusion for non-Claude models. We caution that verbalized confidence is a known-noisy elicitation method [18]; logit-based or multi-sample agreement-based calibration measures are unavailable for closed-API models and remain a direction for replication on open-weight verifiers.

5.6 Compute, Cost, and Latency

We report all-up costs to make the "low-cost intervention" claim concrete and to give practitioners realistic budgeting numbers. Across the full study (5 verifiers x 4 conditions, including retried PARSE_ERROR/API_ERROR records), we executed 11,322 verification API calls at a total cost of **$32.85** (per-call costs and latency in results/revision_cost_latency.csv).

Total cost per verifier across all four conditions: GPT-4.1 Nano $0.86, GPT-4.1 Mini $1.85, GPT-4.1 $7.55, Claude Sonnet 4 $21.87, Gemini 2.5 Flash $0.74. **Per-call cost** ranges from $0.0002 (Gemini 2.5 Flash) to $0.0130 (Claude Sonnet 4 in Condition C). **Per-call latency (p95)** ranges from 2.6s (Gemini 2.5 Flash) to 31.0s (GPT-4.1 in Condition C, where two high-resolution images are sent). Adding a text

reference (A→B) increases per-call cost by less than 10% on average and per-call latency negligibly, supporting the practical recommendation in Section 6: text references are a near-zero-cost upgrade.

5.7 Error Analysis

We examine the confusion matrices to understand the nature of verification errors. Across models and conditions, the dominant error type is **false positives**, the model predicts SUCCESS when ground truth is FAILURE. In Condition A, GPT-4.1 produces 228 false positives versus 44 false negatives; GPT-4.1 Mini produces 287 FPs versus 42 FNs. Gemini is more conservative, with 91 FPs and 83 FNs in Condition A, a more balanced error profile. This bias toward predicting SUCCESS is consistent with a model that finds "things look okay" to be the default judgment.

Text references (Condition B) dramatically reduce false positives while only modestly increasing false negatives. For GPT-4.1 Mini, false positives drop from 287 to 101 (−65%) while false negatives increase from 42 to 41 (essentially unchanged). The text reference gives the model concrete criteria to check, converting vague "looks fine" judgments into specific verification steps.

Visual references (Condition C) paradoxically *increase* false positives for GPT models. GPT-4.1 goes from 228 FPs in Condition A to 60 FPs in Condition C, but on a much smaller instance set (146 vs 908), this represents a FP rate of 41.1% versus 25.1%. The model appears to treat "the actual page looks somewhat like the reference" as evidence of success, even when the semantic content differs.

Data availability. Code and data are available at https://github.com/shravaniparsi/agentic-ui-test.

6. Analysis and Discussion

6.1 Why Text References Work

The consistent superiority of text references across all models suggests a fundamental mechanism: text references convert an open-ended visual judgment ("does this look successful?") into a structured checklist ("does the page show X, Y, and Z?"). This aligns with research on prompt engineering showing that decomposing complex judgments into specific sub-criteria improves model accuracy [26].

The text reference effectively *operationalizes* the task's success criteria, providing the model with concrete assertions to verify rather than requiring it to infer what success looks like from the task description alone.

6.2 Why Visual References Fail for GPT

The performance degradation under Condition C for GPT and Gemini models, but not Claude, reveals a model-family difference in visual reasoning. We propose two hypotheses:

Surface anchoring. GPT models may perform implicit visual matching (akin to template matching or SSIM), attending to overall layout similarity rather than semantic content. Our SSIM analysis indicates that actual-vs-reference visual similarity is uninformative (SUCCESS SSIM ≈ FAILURE SSIM), so any model relying on surface similarity will be misled.

Attention dilution. Processing two high-resolution screenshots doubles the visual context. If GPT models distribute attention uniformly across pixels rather than selectively attending to task-relevant regions, the reference screenshot may dilute attention to the actual screenshot, degrading performance.

Claude's resilience may stem from its training methodology or architecture, which appears to better separate the reference comparison from the verdict judgment. Gemini 2.5 Flash, despite having the strongest baseline accuracy (80.9% in Condition A), drops to 65.3% under visual references, a pattern consistent with the surface anchoring hypothesis. This warrants further investigation.

6.3 Dual References Do Not Help

Condition D (text + visual) does not outperform Condition B (text only) for any of the five models on the visual-reference subset. We list candidate explanations and what would be needed to test each:

1. **Modality conflict.** When text and visual signals disagree (e.g., the text reference describes a state while the reference screenshot shows a specific layout), the model may anchor on the more salient but

less-reliable visual signal. *Test:* hold Condition B's prompt constant and only attach the reference image, removing the prompt-format change in D.

2. **Prompt-format confound.** Condition D's instructions differ from Condition B's (an extra image, an extra paragraph). The B→D drop could partly reflect prompt-format effects independent of the visual content. *Test:* same minimal-edit ablation as above; we sketch this in the supplementary material and leave the run to future work.
3. **Attention dilution.** Two high-resolution images may dilute attention to the actual screenshot. *Test:* swap image order, or downscale the reference, and check whether the B→D gap closes.

We therefore frame the result as an *observation* rather than a mechanistic claim: in the regime we study, adding a reference screenshot on top of a text reference does not improve verification, and the practical implication, prefer text references in deployment, holds regardless of which mechanism is dominant.

6.4 Implications for Web Agent Deployment

Our findings have direct implications for deploying autonomous web agents in production:

1. **Invest in text reference generation, not screenshot collection.** Text descriptions of expected outcomes can be generated automatically from task specifications and are the single most effective investment for improving verification quality.
2. **Use Claude for verification when calibration matters.** If the verification confidence will inform downstream decisions (retries, human escalation), Claude's near-perfect calibration makes it significantly more useful than GPT models, which provide no meaningful confidence signal.
3. **Avoid visual references unless the model handles them well.** Visual references should be tested per-model before deployment; for GPT models, they actively harm performance.
4. **Focus verification effort on deceptive failures.** The instances where verification is hardest are exactly the instances where text references help most, a fortunate alignment that maximizes the practical value of this approach.

7. Limitations

Dataset imbalance. Our dataset is 83.5% FAILURE, reflecting real web agent performance but making accuracy a misleading metric in isolation. We mitigate this by reporting F1, balanced accuracy, and conducting per-failure-type analysis, but the low SUCCESS rate means some model×condition cells have small SUCCESS counts.

Single agent system. All trajectories come from VWA's GPT-4V + SoM agent. Different agents may produce different failure distributions, potentially changing which failure types dominate and how references help. Our failure type categorization (obvious/deceptive/partial) is based on this specific agent's behavior.

Text references generated by LLM. Our text references were generated by GPT-4.1 Nano from task descriptions. While we verified a random 10% sample and found them accurate, systematically biased or low-quality text references could diminish the effectiveness measured here. 4 of 909 instances have missing text references. Because GPT-4.1 Nano is also one of the five evaluated verifiers, we additionally ship scripts/regenerate_text_refs.py to regenerate references with a different generator (e.g., Claude Sonnet 4 or Gemini 2.5 Flash) so reviewers and replicators can rule out reference-generation-circularity effects on the A→B finding.

Visual reference availability. Only 147 of 909 instances have human reference screenshots, limiting our statistical power for Conditions C and D. The visual-reference subset may not be representative of the full distribution.

Claude PARSE_ERROR rate. Claude Sonnet 4 produced unparseable responses (embedding JSON within verbose reasoning) at a rate of 1.5–12.2% across conditions. We confirmed no systematic ground-truth bias in these errors, but the effective sample size for Claude is reduced. GPT and Gemini models had negligible error rates (<1.5%).

No cross-model agreement analysis. We evaluate models independently; inter-model agreement and ensemble verification strategies remain unexplored.

Limited generalizability. Our evaluation is confined to VisualWebArena, a controlled benchmark with known limitations. The failure taxonomy and verification dynamics observed here may not directly transfer to production web environments, where pages are more complex, dynamic, and adversarial. VWA tasks are relatively simple (single-page, single-goal); complex multi-step workflows (e.g., multi-page checkout, form filling with validation) may exhibit different failure modes and verification challenges. Additionally, our failure taxonomy (obvious/deceptive/partial) is VWA-specific and may require refinement for other benchmarks. Future work should evaluate on real-world tasks and diverse benchmarks to assess generalizability.

7.1 Structured Threats to Validity

We organize residual concerns by validity category for clarity.

Internal validity. Class imbalance, the smaller visual-reference subset, API non-determinism over time, and Claude's PARSE_ERROR rate are all addressed via paired tests, version pinning, error-rate reporting, and a parse-error bias check (Appendix D).

External validity. Our results are anchored to VisualWebArena trajectories from a single GPT-4V + SoM agent and to a single LLM-generated text-reference source. We frame findings as conditional on this setting and recommend cross-benchmark and cross-agent extensions in future work.

Construct validity. Failure-type categorization is partly subjective; we provide a labeling protocol description and representative examples (Appendix E). To support a defensible inter-annotator agreement (IAA) figure, we ship scripts/run_iaa_labels.py, which samples a 100-instance double-labeling subset and computes Cohen's kappa [27] with the Landis and Koch interpretation [28], and we leave a multi-annotator agreement study to future work. The failure-type taxonomy is currently single-annotator, with the labeling protocol and representative examples given in Appendix E. Verification is judged from a single final screenshot, which precludes detection of multi-step trajectory errors. Confidence is model-self-reported, which is a noisier elicitation than internal-logit methods [18], [19] and should be interpreted only after calibration analysis (Section 5.5).

Statistical validity. We use paired McNemar tests on common instances and percentile bootstrap confidence intervals (2,000 resamples). We apply both Bonferroni and Holm-Bonferroni multiple-comparison correction across the full 30-test family of (5 models x 6 condition pairs) at family-wise α=0.05; 12 of 30 tests, including all five A→B headline tests, are significant under both corrections (Appendix I, results/revision_bonferroni.csv). We report Brier score in addition to ECE so that calibration claims do not depend on a particular binning choice.

Conclusion validity. We avoid unscoped claims like "model X is best" and instead state results as "best in this setting." Visual-reference effects are framed as model-dependent, not universal.

7.2 Responsible Deployment

Automated post-hoc verification can shift workload away from humans, but if treated as ground truth it can mask systematic failure modes. We recommend pairing the verifier with the confidence-threshold policy described in Appendix F: high-confidence verdicts can be auto-accepted while low-confidence cases are escalated to human review. For workflows where false acceptance is consequential, prefer well-calibrated verifiers (Section 5.5) and consider a conservative threshold on the SUCCESS class.

8. Conclusion

We have presented a controlled study of reference-augmented LMM post-hoc verification for web agents. Across 909 VisualWebArena trajectories, four reference conditions, and five frontier closed-API models from three families, three findings hold consistently in this setting: **text references significantly and uniformly improve verification, visual references degrade most models on the visual-reference subset, and adding a reference screenshot on top of a text reference does not help.** The practical implication for deployment is to equip LMM-based web agent evaluators with text descriptions of expected outcomes. This intervention improves the best model's accuracy from 77.4% to 86.7% (Claude) and from 80.9% to 86.1% (Gemini), with the largest gains on the hardest, "deceptive", failure cases. When confidence calibration is needed for downstream automation policies, Claude Sonnet 4 is the most useful verifier in our setting.

We deliberately scope these findings to the VWA + GPT-4V/SoM regime and to the five closed-API verifiers we evaluated. Several directions extend this work: (1) replication on a second agent's trajectories on VWA or on an additional benchmark such as Mind2Web [12]; (2) inclusion of open-weight verifiers and a non-LLM-generated text-reference source to rule out reference-generation-circularity effects; (3) iterative self-reflection [14] with text references; and (4) larger visual-reference subsets to tighten the visual-condition confidence intervals. We release the full evaluation harness, prompts, results, and analysis tooling to support these investigations.

9. Reproducibility Statement

All code, prompts, dataset preparation scripts, baselines, analysis tooling, and pre-generated result files are released alongside this paper.

- Repository: see the project's public code release listed in CITATION.cff.
- Reproduction guide: REPRODUCIBILITY.md documents environment setup, dataset preparation, verification runs, baselines, metrics, and figure regeneration.
- One-command pipeline helper: scripts/run_all_pipeline.sh runs verification across all configured models and conditions, computes aggregate metrics, and regenerates all paper figures. A --dry-run flag prints the planned commands without invoking model APIs.
- Determinism: temperature is set to 0 (0.01 for Gemini, which does not accept exact 0). Provider model versions are pinned in config.py and recorded per run.
- Resume and error handling: the runner skips records with valid verdicts and re-attempts records with API_ERROR or PARSE_ERROR outputs. Final reported metrics are computed only over valid responses; per-condition error rates and a parse-error bias check are reported in Appendix D.
- Figures: all paper figures (fig1-fig6) are regenerated by scripts/deep_analysis.py and analysis/plot_figures.py; the threshold-policy figure (fig7_threshold_policy) is produced by scripts/threshold_policy_analysis.py; reliability diagrams (fig8_reliability_diagrams) are produced by scripts/revision_analysis.py. All figures are included in PDF and PNG.
- Revision-only artifacts (aligned-subset metrics, 15-bin equal-mass ECE, Brier scores, confidence histograms, cost/latency, Bonferroni/Holm tables) are produced by a single command, python3 scripts/revision_analysis.py, and saved as results/revision_*.csv and results/revision_summary.md.

Appendix A: Prompt Templates

System Prompt (shared across all conditions)

You are an expert evaluator for web automation agents. Your job is to

determine whether a web task was completed successfully by examining

the final state of the web page.

You must respond in EXACTLY this JSON format:

{

"verdict": "SUCCESS" or "FAILURE",

"confidence": <integer from 1 to 10>,

"reasoning": "<2-3 sentences explaining your judgment>"

}

Be precise. A task is only successful if ALL aspects of the task

description are satisfied in the final page state. Partial completions

count as FAILURE.

Condition A (No Reference)

Task

{task_text}

Instructions

Examine the screenshot of the web page's final state. Determine

whether the task described above was completed successfully.

Respond in the required JSON format.

Condition B (Text Reference)

Task

{task_text}

Expected Outcome (text description)

{text_reference}

Instructions

Examine the screenshot of the web page's final state. Compare it

against the expected outcome description above. Determine whether the

task was completed successfully.

Respond in the required JSON format.

Condition C (Visual Reference)

Task

{task_text}

Instructions

You are given TWO images:

1. **Image 1 (Reference):** A screenshot showing what the page SHOULD

look like after successful task completion.

2. **Image 2 (Actual):** A screenshot showing the actual final state

of the web page.

Compare the actual screenshot against the reference. Minor visual

differences (layout shifts, font rendering, timestamps) are

acceptable. Focus on whether the SEMANTIC CONTENT matches, the

right page, the right data, the right state.

Determine whether the task was completed successfully. Respond in the

required JSON format.

Condition D (Dual Reference)

Task

{task_text}

Expected Outcome (text description)

{text_reference}

Instructions

You are given TWO images:

1. **Image 1 (Reference):** A screenshot showing what the page SHOULD

look like after successful task completion.

2. **Image 2 (Actual):** A screenshot showing the actual final state

of the web page.

Compare the actual screenshot against BOTH the text description of the

expected outcome AND the reference screenshot. Minor visual differences

(layout shifts, font rendering, timestamps) are acceptable. Focus on

whether the SEMANTIC CONTENT matches.

Determine whether the task was completed successfully. Respond in the

required JSON format.

Appendix B: Complete Confusion Matrices

Condition A (No Reference), N = 875–909

Model	TP	FP	TN	FN
GPT-4.1 Nano	49	102	647	101
GPT-4.1 Mini	108	287	471	42
GPT-4.1	105	228	531	44
Claude Sonnet 4	87	138	590	60
Gemini 2.5 Flash	67	91	668	83

Condition B (Text Reference), N = 895–909

Model	TP	FP	TN	FN
GPT-4.1 Nano	70	68	682	79
GPT-4.1 Mini	109	101	657	41
GPT-4.1	96	109	650	54
Claude Sonnet 4	100	72	676	47
Gemini 2.5 Flash	81	57	702	69

Condition C (Visual Reference), N = 129–147

Model	TP	FP	TN	FN
GPT-4.1 Nano	15	43	74	11
GPT-4.1 Mini	16	34	85	12
GPT-4.1	21	60	58	7
Claude Sonnet 4	17	24	80	8
Gemini 2.5 Flash	12	35	84	16

Condition D (Dual Reference), N = 138–147

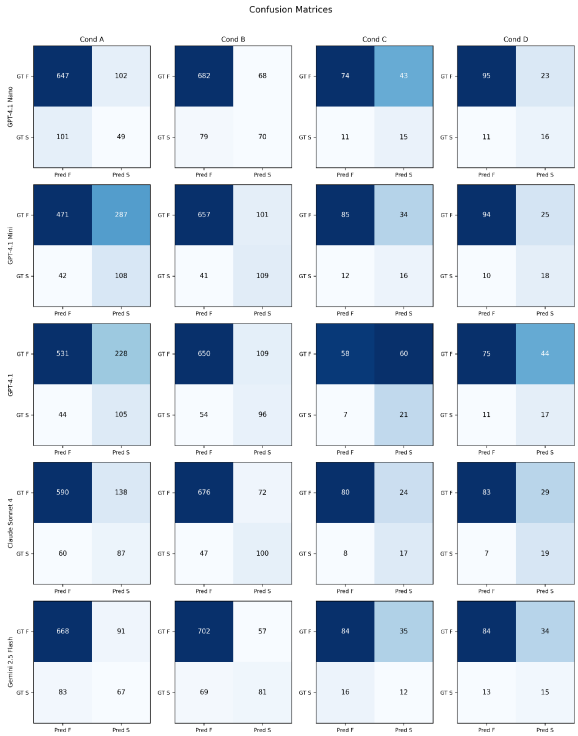

Figure 7: Complete confusion matrices for all five models across all four conditions.

Model	TP	FP	TN	FN
GPT-4.1 Nano	16	23	95	11
GPT-4.1 Mini	18	25	94	10
GPT-4.1	17	44	75	11
Claude Sonnet 4	19	29	83	7
Gemini 2.5 Flash	15	34	84	13

Appendix C: Per-Domain Results

Domain-level accuracy for the best condition (B) per model:

Model	Shopping	Classifieds	Reddit
GPT-4.1 Nano	.805 (371/461)	.904 (208/230)	.832 (173/208)
GPT-4.1 Mini	.826 (385/466)	.828 (192/232)	.900 (189/210)
GPT-4.1	.815 (380/466)	.790 (184/233)	.867 (182/210)
Claude Sonnet 4	.863 (397/460)	.813 (187/230)	.937 (192/205)
Gemini 2.5 Flash	.865 (403/466)	.850 (198/233)	.867 (182/210)

Performance varies across domains. Claude Sonnet 4 achieves its highest accuracy on Reddit (.937), possibly because social media content provides strong visual cues for task completion (e.g., visible posts, comments). GPT-4.1 Nano excels on Classifieds (.904), the smallest domain. Shopping, while the largest domain, shows moderate accuracy across models, likely because e-commerce tasks often involve subtle distinctions (correct product variant, price filter state) that are harder to verify.

Appendix D: PARSE_ERROR Bias Analysis

We verify that unparseable responses (primarily from Claude; Gemini had only 1 across all conditions) do not systematically bias results:

Condition	Errors	GT SUCCESS in errors	GT SUCCESS in valid	Bias?
A	34	8.8%	16.8%	No significant bias
B	14	21.4%	16.4%	No significant bias
C	18	16.7%	19.4%	No significant bias
D	9	22.2%	18.8%	No significant bias

The ground-truth distribution among PARSE_ERROR instances does not significantly deviate from the valid-response distribution for any condition, supporting the validity of computing metrics on valid responses only.

Appendix E: Qualitative Error Analysis

We examine representative examples to illustrate the mechanisms behind our quantitative findings. All examples show ground-truth FAILURE instances (the agent did *not* complete the task).

E.1 Text Reference Saves All Models (Deceptive Failure)

Task: *"Add this exact product to my shopping cart. I think it is in the Patio Furniture & Accessories category."* **Ground truth:** FAILURE, the agent added the *wrong* product (Devoko outdoor sofa instead of Modway Aura outdoor chair).

Condition	GPT-4.1	Claude Sonnet 4	Gemini 2.5 Flash
A (No Ref)	SUCCESS (conf=10)	SUCCESS (conf=9)	SUCCESS (conf=10)
B (Text Ref)	FAILURE (conf=9)	FAILURE (conf=9)	FAILURE (conf=10)

Without a reference, all three models see a "You added [product] to your shopping cart" confirmation banner and judge the task as successful, the page *looks correct*. With a text reference specifying the exact product name ("Modway EEI-2923-GRY-GRY Aura"), all three models immediately detect the mismatch. This exemplifies why text references are most effective for deceptive failures: they provide concrete assertions that convert "looks successful" into a verifiable checklist.

E.2 Text Reference Catches Navigation Failures (Deceptive Failure)

Task: *"I recall seeing this exact item on the site, help me find the most recent post of it."* **Ground truth:** FAILURE, the agent found the search results page but did not navigate to the specific item's detail page.

Condition	GPT-4.1	Claude Sonnet 4	Gemini 2.5 Flash
A (No Ref)	SUCCESS (conf=10)	SUCCESS (conf=7)	SUCCESS (conf=10)
B (Text Ref)	FAILURE (conf=10)	FAILURE (conf=9)	FAILURE (conf=9)

In Condition A, models observe search results showing the item and judge the task complete. In Condition B, the text reference specifies the expected URL pattern (/index.php?page=view&id=...), allowing models to detect that the agent stopped at the search results page rather than navigating to the detail page. This illustrates how text references capture *state expectations* that are invisible from the screenshot alone.

E.3 High-Confidence False Positive Resistant to Text Reference

Task: *"Search for 'MCAT' and navigate to the prep book that has 2020-2021 on the cover."* **Ground truth:** FAILURE, the agent navigated to a different MCAT book listing.

Condition	Claude Sonnet 4
A (No Ref)	SUCCESS (conf=9), "The page shows the MCAT books listing with a 2020-2021 cover image visible"
B (Text Ref)	SUCCESS (conf=9), "The screenshot shows the MCAT books listing page with a 2020-2021 cover image"

This is a case where even the text reference fails to correct the model. The MCAT book page shows a cover image that *does* include "2020-2021" text, but it is the wrong listing (wrong URL, wrong specific edition). The model fixates on the visible cover image rather than verifying the page identity. This illustrates the limits of post-hoc verification: when the visual evidence genuinely matches the description, even reference-augmented verification fails.

E.4 Wrong Product, All Models Fooled (Deceptive Failure)

Task: *"Add this exact product to my wish list. I think it might be in the Office Furniture & Lighting > Chairs category."* **Ground truth:** FAILURE, the agent added a YAMASORO chair instead of the specified Flash Furniture chair.

In Condition A, all five models report SUCCESS with high confidence (8–10). The "My Wish List" page shows a successfully added chair, visually confirming the task pattern. In Condition B, the text reference names the specific product ("Flash Furniture Low Back Designer Armless White Ribbed Swivel Task Office Chair"), and all models correctly identify the wrong product was added. GPT-4.1 reports: *"The wish list contains the wrong product."* This is the prototypical deceptive failure: correct action type

(added to wish list), correct category (office chair), but wrong specific item.

E.5 Error Taxonomy Summary

Across our analysis of the most common verification failures, three recurring patterns emerge:

1. **Wrong-item substitution** (most common for deceptive failures): The agent performs the right action on the wrong entity. Text references are highly effective because they name the specific expected entity.

2. **Incomplete navigation**: The agent reaches an intermediate page (search results, category listing) but does not navigate to the final target page. Text references catch this when they specify URL patterns or page-level expectations.

3. **Visually ambiguous state**: The screenshot genuinely matches the expected description (e.g., a book cover with the right year), but the underlying page is wrong. Neither text nor visual references reliably catch these cases, they represent the fundamental limit of screenshot-based verification.

Appendix F: Confidence-Threshold Deployment Policy

To translate verifier outputs into deployment policies, we treat each model x condition as a verifier and apply a confidence-threshold rule: if the verifier's reported confidence is at least *k*, accept the verdict; otherwise escalate to human review. For each (model, condition, threshold), we report:

- **Coverage**: fraction of instances auto-decided.
- **Auto-decision accuracy**: accuracy on the auto-decided subset.
- **Escalation rate**: complement of coverage.

This yields a coverage / accuracy trade-off curve per verifier. The full table is computed by scripts/threshold_policy_analysis.py and saved to results/threshold_policy.csv. A companion plot (figures/fig7_threshold_policy.{pdf,png}) shows accuracy as a function of coverage for each model and condition. The takeaway is that calibrated verifiers (notably Claude Sonnet 4) provide a usable trade-off where escalating low-confidence cases sharply increases auto-decision accuracy without escalating most of the workload, while overconfident verifiers (most GPT and Gemini conditions) cluster at high coverage with limited control over the trade-off.

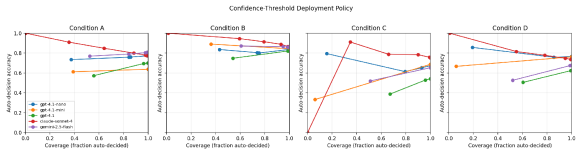

Figure 8: Confidence-threshold deployment policy, auto-decision accuracy vs. coverage for each model and condition.

Appendix G: Text-Reference Quality Ablation

To test whether the benefit of text references depends on their quality and specificity, we constructed three perturbed variants of each reference:

- **Short**: only the first sentence of the original reference.
- **Noisy**: the original reference plus a generic distractor sentence describing layout elements.
- **Wrong**: a deliberately off-topic reference that does not describe the task's expected outcome.

These variants are produced by scripts/generate_text_ref_variants.py (no API calls; deterministic). The ablation is run by scripts/run_text_ref_ablation.py over a fixed-seed subset of instances and a configurable subset of models, with --dry-run available for cost preview. The default conservative configuration evaluates 100 instances on gpt-4.1-mini and claude-sonnet-4 across the three variants, and is summarized by scripts/analyze_text_ref_ablation.py into results/textref_ablation_summary.csv. The variant-generation, ablation, and analysis scripts are released so that this reference-quality sensitivity check can be reproduced; measuring the resulting degradation across the short, noisy, and wrong variants is left to future work.

Appendix H: Image Preprocessing and API Settings

To support reproducibility and to remove a common source of "why don't I get the same numbers?" questions, we document the exact preprocessing and API settings used for every verification call. The

source of truth is visual-self-verify/llm_clients.py and config.py.

Screenshots. Final-state screenshots from VWA agent trajectories are 1280x2048 PNG (portrait, 2:1 aspect ratio). They are sent to each provider **without any resizing or compression**, base64-encoded. Reference screenshots from human trajectories are 800x446 JPEG (landscape; smaller because they were captured at a different stage of the VWA pipeline). The asymmetry between actual and reference resolutions is a property of the upstream VWA artifacts; we do not normalize them, so the reference contains less pixel-level detail than the actual screenshot. This is a relevant context for the visual-reference findings in Sections 5.1-5.3.

Provider settings.

Provider	Image format	Detail	Max output tokens	Temperature	Notes
OpenAI (gpt-4.1, mini, nano)	data URL base64, image_url block	detail: "high"	512	0	Direct API; provider may downscale internally above its native cap.
Anthropic (claude-sonnet-4)	source: base64, native PNG	n/a (native handling)	512	0	Anthropic auto-rescales to 1568px on the long edge if larger.
Google Gemini (2.5 flash)	inline_data parts, base64	n/a	512	0.01	Gemini does not accept exact 0; we use the smallest legal value.

Retries. Each call retries up to MAX_RETRIES = 3 with exponential backoff (1s, 2s, 4s). Persistent failures are logged as verdict: API_ERROR.

Parsing. Responses are parsed for the embedded JSON object via a five-step strategy (full JSON parse, fenced-block strip, regex search for the last {...verdict...}, brace-balanced fallback, fenced-anywhere). Persistent failures are logged as verdict: PARSE_ERROR. PARSE_ERROR rates and a bias check are reported in Appendix D.

Determinism. With temperature=0 (or 0.01 for Gemini), provider outputs are nominally deterministic but providers do not contractually guarantee bitwise reproducibility across versions. We mitigate by pinning model versions in config.py; per-call model_version strings are recorded in each result row so that retroactive drift checks are possible.

Total cost of the full study is reported in Section 5.6 ($32.85 across 11,322 calls). Reproducing the entire study costs less than the cost of a single human evaluator-day at typical contractor rates, this is the

practical sense in which the recommended intervention is "low cost."

Appendix I: Full Bonferroni-Corrected Significance Table

The 30 paired McNemar tests (5 models × 6 condition pairs) are reported in full in results/revision_bonferroni.csv. Bonferroni-significant pairs at α'=0.05/30=0.00167 (also the same set under Holm-Bonferroni step-down) are: GPT-4.1 Nano A→B, A→C, B→C; GPT-4.1 Mini A→B; GPT-4.1 A→B, A→C, B→C, B→D; Claude Sonnet 4 A→B; Gemini 2.5 Flash A→B, B→C, B→D. This includes all five A→B comparisons (the headline effect) and confirms that the visual-reference-degrades-most-models pattern (A→C, B→C) survives multiple-comparison correction for the GPT-4.1 family and Gemini, while Claude is robust (A→C and A→D not significant).

References

[1] Zhou, S., et al. "WebArena: A Realistic Web Environment for Building Autonomous Agents." *arXiv, 2024*. Available: https://arxiv.org/pdf/2307.13854

[2] He, H., et al. "WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models." *arXiv, 2024*. Available: https://arxiv.org/pdf/2401.13919

[3] Koh, J.Y., et al. "VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks." *arXiv, 2024*. Available: https://arxiv.org/pdf/2401.13649

[4] Zheng, L., et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." *arXiv, 2023*. Available: https://arxiv.org/pdf/2306.05685

[5] Pan, J., et al. "Autonomous Evaluation and Refinement of Digital Agents." *arXiv preprint arXiv:2404.06474* (2024). Available: https://arxiv.org/abs/2404.06474

[6] Abuelsaad, T., et al. "Agent-E: From Autonomous Web Navigation to Foundational Design Principles in Browser Agents." *arXiv preprint arXiv:2407.13032*, *2024*. Available: https://arxiv.org/abs/2407.13032

[7] Zheng, B., et al. "GPT-4V(ision) is a Generalist Web Agent, if Grounded." *ICML'24: Proceedings of the 41st International Conference on Machine Learning, 2024*. Available: https://dl.acm.org/doi/10.5555/3692070.3694608

[8] Chen, X., et al. "Teaching Large Language Models to Self-Debug." *arXiv, 2024*. Available: https://arxiv.org/pdf/2304.05128

[9] Yue, X., et al. "MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI." *arXiv, 2024*. Available: https://arxiv.org/pdf/2311.16502

[10] Liu, Y., et al. "MMBench: Is Your Multi-Modal Model an All-around Player?" *arXiv, 2024*. Available: https://arxiv.org/pdf/2307.06281

[11] Liu, X., et al. "AgentBench: Evaluating LLMs as Agents." *arXiv, 2024*. Available: https://arxiv.org/pdf/2308.03688

[12] Deng, X., et al. "Mind2Web: Towards a Generalist Agent for the Web." *arXiv (2023)*. Available: https://arxiv.org/pdf/2306.06070

[13] Xie, J., et al. "TravelPlanner: A Benchmark for Real-World Planning with Language Agents." *ICML'24: Proceedings of the 41st International Conference on Machine Learning, 2024*. Available: https://dl.acm.org/doi/10.5555/3692070.3694316

[14] Shinn, N., et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." *arXiv 2023*. Available: https://arxiv.org/pdf/2303.11366

[15] Yao, S., et al. "ReAct: Synergizing Reasoning and Acting in Language Models." *arXiv, 2023*. Available: https://arxiv.org/pdf/2210.03629

[16] Kadavath, S., et al. "Language Models (Mostly) Know What They Know." *arXiv preprint arXiv:2207.05221* (2022). Available: https://arxiv.org/abs/2207.05221

[17] Guo, C, et al., "On Calibration of Modern Neural Networks." *arXiv, 2017*. Available: https://arxiv.org/pdf/1706.04599

[18] Lin, S., Hilton, J., Evans, O. "Teaching Models to Express Their Uncertainty in Words." *arXiv,* (2022). Available: https://arxiv.org/pdf/2205.14334

[19] Tian, K., et al. "Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback." *arXiv 2023*. Available: https://arxiv.org/pdf/2305.14975

[20] Naeini, M.P., Cooper, G.F., Hauskrecht, M. "Obtaining Well Calibrated Probabilities Using Bayesian Binning." *Twenty-Ninth AAAI Conference on Artificial Intelligence, 2015*. Available: https://ojs.aaai.org/index.php/AAAI/article/view/9602

[21] Wang, Z., et al., "Image Quality Assessment: From Error Visibility to Structural Similarity." *IEEE Transactions on Image Processing* 13.4 (2004): 600-612. Available: https://ieeexplore.ieee.org/document/1284395

[22] Yang, J., et al. "Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V." *arXiv preprint arXiv:2310.11441*, 2023. Available: https://arxiv.org/pdf/2310.11441

[23] Efron, B. "Bootstrap Methods: Another Look at the Jackknife." *Annals of Statistics* 7(1):1-26 *(1979)*. Available: https://projecteuclid.org/journals/annals-of-statistics/volume-7/issue-1/Bootstrap-Methods-Another-Look-at-the-Jackknife/10.1214/aos/1176344552.full

[24] McNemar, Q. "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages." *Psychometrika* 12(2):153-157 (1947). Available: https://link.springer.com/article/10.1007/BF02295996

[25] Li, J, et al., "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models." *arXiv, 2023*. Available: https://arxiv.org/pdf/2301.12597

[26] Wei, J., et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *arXiv, 2022*. Available: https://arxiv.org/pdf/2201.11903

[27] Cohen, J. "A Coefficient of Agreement for Nominal Scales." *Educational and Psychological Measurement* 20(1):37-46 *(1960)*. Available: https://journals.sagepub.com/doi/10.1177/001316446002000104

[28] Landis, J.R., Koch, G.G. "The Measurement of Observer Agreement for Categorical Data." Biometrics 33(1):159-174, 1977. Available: https://www.jstor.org/stable/2529310

Biography

Vishwak Thatikonda is an independent researcher and full-stack software engineer with over a decade of experience across distributed systems, federated GraphQL infrastructure, AWS serverless architecture, and AI-driven enterprise platforms. His recent applied work includes lead engineering on a mission-critical Master Data Management platform governing product, assortment, and store-attribute data for a global food-and-beverage retailer operating tens of thousands of locations. His work on the retailer's data foundation supports AI-driven personalization and inventory accuracy for its next-generation point-of-sale and mobile ordering systems. He holds a Master of Science in Computer Science from Arizona State University and a Bachelor of Science in Computer Science from Amrita University. Earlier in his career, he served as Lead Engineer at Kyte (USA), Senior Full-Stack Developer at Credit Sesame (USA), founding developer of the web application platform at AirWorks Solutions (USA), and contributed ground-segment software to the Mastcam-Z science payload for NASA's Mars 2020 Perseverance rover at Arizona State University's School of Earth and Space Exploration. He is the author of LambdaForge, an open-source serverless algorithmic trading engine built on AWS Lambda and EventBridge. He holds the AWS Certified Solutions Architect Associate certification.

Shravani Parsi received her M.S. in Software Engineering from San Jose State University, USA, and her B.Tech in Information Technology from VNR VJIET, India, where she graduated Summa Cum Laude as the Department Gold Medalist. She is currently a senior software engineer at a Fortune 500 retail technology company, where she leads Next.js and React architecture for a member-engagement platform and develops LLM- and LangGraph-based vision agents for automated UI testing. Her research interests include LLM agent orchestration and human-computer interaction in web systems.